

Week01 实验报告

mrw1114

August, 27th

1 个人信息

- 姓名：王俊杰
- 学号：25020007122
- Github 仓库链接：<https://github.com/mrw1114/fundamentals-of-system-dev-tools>

2 课上实验

2.1 第 01 题：含空格文件名的批量整理

2.1.1 创建 input 目录和对应的文件

创建目录使用 `mkdir` 命令；随后利用 `echo` 命令新建并写入文件，其中 `-e` 选项启用转义字符识别，`-n` 选项不输出换行，同时使用单引号包裹文件名确保空格被识别成文件名的一部分；接下来利用 `touch` 命令新建空文件；`run.log` 中的日志为模拟的两条日志。接下来用 `ls` 命令展示目录结构，其中 `-alR` 选项分别代表显示隐藏文件、列出详细信息、递归地显示目录及子目录内容；之后使用 `cat` 命令展示各文件内容

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ mkdir ./input/
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ mkdir ./input/docs
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ mkdir ./input/tmp
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ echo -e -n "alpha\nbeta\n" > './input/docs/note one.txt'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ echo 'hidden' > './input/docs/.secret.txt'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ touch './input/tmp/empty.txt'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ echo -e -n "09:00:00,process started\n09:02:22,process terminated\n" > './input/run.log'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ ls -alR ./input
./input:
总计 20
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 3 mrw mrw 4096 Aug 31 00:01 ..
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 docs
-rw-rw-r-- 1 mrw mrw 53 Aug 31 00:04 run.log
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 tmp

./input/docs:
总计 16
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 ..
-rw-rw-r-- 1 mrw mrw 11 Aug 31 00:03 'note one.txt'
-rw-rw-r-- 1 mrw mrw 7 Aug 31 00:04 .secret.txt

./input/tmp:
总计 8
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 ..
-rw-rw-r-- 1 mrw mrw 0 Aug 31 00:04 empty.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cat './input/docs/note one.txt'
alpha
beta
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cat './input/docs/.secret.txt'
hidden
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cat './input/tmp/empty.txt'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cat './input/run.log'
09:00:00,process started
09:02:22,process terminated
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$

```

图 1: 新建操作及效果

2.1.2 显示绝对路径和 input 下全部文件

使用 `realpath` 命令来获取 q01 的真实路径，再用 `ls -alR` 命令列出所有文件及隐藏文件

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ realpath .
/home/mrw/sys-dev-test/q01
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ ls -alR ./input
./input:
总计 20
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 3 mrw mrw 4096 Aug 31 00:01 ..
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 docs
-rw-rw-r-- 1 mrw mrw 53 Aug 31 00:04 run.log
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 tmp

./input/docs:
总计 16
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 ..
-rw-rw-r-- 1 mrw mrw 11 Aug 31 00:03 'note one.txt'
-rw-rw-r-- 1 mrw mrw 7 Aug 31 00:04 .secret.txt

./input/tmp:
总计 8
drwxrwxr-x 2 mrw mrw 4096 Aug 31 00:04 .
drwxrwxr-x 4 mrw mrw 4096 Aug 31 00:04 ..
-rw-rw-r-- 1 mrw mrw 0 Aug 31 00:04 empty.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$

```

图 2: 真实路径与全部文件

2.1.3 复制 txt 文件并保留相对目录结构

首先新建目录 work25020007122。由于需要对目录中所有 txt 文件进行复制并保留相对结构，不能直接使用 cp 命令递归复制（会把 run.log 也一起进行复制）。可以用 find 命令和通配符查找 input 目录下的每个 txt 文件再利用 cp 命令的 -parents 选项保留路径结构；其中 -type 制定查找文件的类型，f 为普通文件，d 为目录；-exec 表示需对被查找到的文件执行的命令，{} 为查找到的文件名传入的地方；\; 表示经过转义的分号，代表命令的结束，转义是为了避免被解释为行内命令的分隔符。为确保复制的是相对路径，需要用 cd 命令进入 input 文件夹进行复制。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ mkdir ./work25020007122
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cd ./input
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01/input$ find ./ -name "*.txt" -type f -exec cp --parents {} \;
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01/input$ cd ../
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$

```

图 3: 选择性复制并保留相对结构

2.1.4 修改权限并生成总结

利用 find 命令查找 work25020007122 中的普通文件和目录，分别执行 chmod 750 和 chmod 640 以设置权限。再通过 find 命令的 -printf 选项生成统计总结，其中%p 表示含相对目录的文件名，%s 表示文件字节数，再将得到的统计信息重定向到 inventory.txt 文件中。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ chmod 750 ./work25020007122
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ find ./work25020007122 -type d -exec chmod 750 {} \;
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ find ./work25020007122 -type f -exec chmod 640 {} \;
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ find ./work25020007122 -type f -printf "%P %s\n" > inventory.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$ cat ./inventory.txt
docs/note one.txt 11
docs/.secret.txt 7
tmp/empty.txt 0
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q01$

```

图 4: 修改权限与生成总结

2.2 第 02 题：随机访问日志统计

2.2.1 创建 access.csv 文件

将测试数据写入 access.csv 文件。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ echo -e -n 'timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120\n09:00:02,bob,/api/orders,500,310\n09:00:03,alice,/api/users,503,180\n09:00:
04,carol,/login,500,90\n09:00:05,bob,/api/orders,502,260\n09:00:06,dave,/health,200,20\n09:00:07,alice,/api/or
ders,500,420\n09:00:08,carol,/api/users,500,150\n' > access.csv
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ cat access.csv
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$

```

图 5: 写入 access.csv 文件

2.2.2 编写 analyse.sh 文件

使用 vim 工具编辑 analyse.sh 文件, 随后使用 chmod 为其添加可执行权限, access.csv 文件内容如下

Listing 1: analyse.sh 的内容

```

1  #!/bin/bash
2  # 判断是否传入1个参数
3  if [ $# -ne 1 ]; then
4      echo "Usage: $0 <csv_file>" >&2
5      exit 1
6  fi
7  # 判断是否存在文件
8  if [ ! -e $1 ]
9  then
10     echo "$1 doesn't exist" >&2
11     exit 2
12 fi
13
14 # 排序并输出5xx次数最多的 path
15 awk -F',' 'NR!=1&&$4>=500 {cnt[$3]++} END {for(i in cnt)
    print cnt[i],i}' $1 | sort -t',' -k1,1nr -k2,2 | awk -F'
    ' '{print $2}' | head -n 2

```

```

16
17 # 计算平均路径
18 awk -F',' 'NR!=1 {sum+=$5; n++} END {if(n != 0) printf "%.2
    f\n", sum / n; else print "0.00"}' $1

```

其中, `[]` 表示 `test` 命令, `!` 表示反转条件, `-e` 判断文件是否存在, `-ne` 表示数值是否不相等。在使用的内置变量中, `$#` 表示传入的参数数量, `$1` 表示传入的第一个参数。此处使用 `awk` 指令进行数据提取与过滤, 其中 `-F` 制定分割符, `NR` 表示当前行的行号, `END` 表示读取完每一行后才执行后面的操作, `$n` (从 1 开始) 表示第 `n` 列内容。`sort` 指令对每行内容排序, `-t` 指定分隔符, 其中 `-k1,1nr` 表示第 1 个排序规则为对第 1 列按数据方式升序排序后反向排序 (降序), `k2,2` 表示第二个排序规则为对第二列的 `path` 按字典序升序排列。`head -n 2` 表示仅输出前两行

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ vim analyse.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ chmod +x ./analyse.sh
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$

```

图 6: 编写 `analyse.sh` 并赋予其可执行权限

2.2.3 运行程序

分别对 `access.csv` 和不存在的文件 `hajimi.csv` 运行程序观察输出

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ ./analyse.sh access.csv
/api/orders
/api/users
193.75
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ ./analyse.sh hajimi.csv
hajimi.csv doesn't exist
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q02$ █

```

图 7: 观察运行结果

2.3 第 03 题：制造、解决并解释一次合并冲突

2.3.1 新建仓库与初始提交

新建文件夹 my-repo，进入该文件夹执行 git init 完成初始提交。安装的 git 使用 master 作为默认名称，故此处用 main 作为主分支名称。随后修改 config.txt 文件，用 git add 命令将文件加入暂存区，再使用 git commit -m 向本地仓库完成初始提交。在提交前需要使用 git config --global 设置 user.name 和 user.email。

```
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03$ mkdir ./my-repo
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03$ cd ./my-repo/
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示： git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示： git branch -m <name>
已初始化空的 Git 仓库于 /home/mrw/sys-dev-test/q03/my-repo/.git/
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git branch -m main
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ echo 'mode=normal' > config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git add ./config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git config --global user.name 'mrw1114'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git config --global user.email 2711181556@qq.com
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git commit -m 'Initail commit'
[main (根提交) 81ce968] Initail commit
1 file changed, 1 insertion(+)
create mode 100644 config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$
```

图 8: 新建仓库与初始提交

2.3.2 创建不同分支与相应提交

利用 git checkout -b 在 main 的基础上创建新分支 feature-a，在该分支下改写 config.txt 为 “mode=safe” 并提交。随后用 git checkout 切换回 main，在此基础上创建并切换到 feature-b 分支，改写 config.txt 为 “mode=fast” 并提交。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git checkout -b feature-a
切换到一个新分支 'feature-a'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ echo 'mode=safe' > config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git add config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git commit -m 'Set mode = safe'
[feature-a 3a975bd] Set mode = safe
1 file changed, 1 insertion(+), 1 deletion(-)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git checkout main
切换到分支 'main'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git checkout -b feature-b
切换到一个新分支 'feature-b'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ echo 'mode=fast' > config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git add config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git commit -m 'Set mode = fast'
[feature-b 4e249fe] Set mode = fast
1 file changed, 1 insertion(+), 1 deletion(-)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$

```

图 9: 创建两个分支并分别提交

2.3.3 冲突的制造与解决

回到 main 分支, 用 git merge 合并 feature-a 成功, 随后合并 feature-b 失败。因为两个分支都基于 main 分支且修改了同一文件的同一行。这里修改 config.txt 并 commit 来解决冲突。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git checkout main
切换到分支 'main'
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git merge feature-a
更新 81ce968..3a975bd
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git merge feature-b
自动合并 config.txt
冲突 (内容): 合并冲突于 config.txt
自动合并失败, 修正冲突然后提交修正的结果。
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git checkout feature-b
config.txt: needs merge
error: 您需要先解决当前索引的冲突
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ echo -e "mode=safe\nnote=reviewed" > config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git add config.txt
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git commit -m "Merge fix: keep mode = safe and add note = reviewed"
[main 836772a] Merge fix: keep mode = safe and add note = reviewed
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$

```

图 10: 制造与解决冲突

2.3.4 查看历史记录

使用 `git status` 查看工作区是否干净, 确认干净后用制定命令查看日志。由于 feature-a 合并到 main 是 fast-forward, 所以 main 直接指向 feature-a 提交。

```
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git status
位于分支 main
无文件要提交, 干净的工作区
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git log --all --graph --decorate --oneline
* 836772a (HEAD -> main) Merge fix: keep mode = safe and add note = reviewed
|
| * 4e249fe (feature-b) Set mode = fast
| * | 3a975bd (feature-a) Set mode = safe
|/
* 81ce968 Initail commit
nrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$
```

图 11: 查看状态及日志

2.4 第 04 题: 修复并构建一页技术说明

模板中用到了中文, 需要使用 `ctex` 宏包进行兼容。由于 `pdflatex` 对中文兼容性差, 故采用 `xelatex` 进行处理。

2.4.1 新建模板文件

用 `vim` 将内容输入到 `report.tex` 中

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test$ cd ./q04
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q04$ vim ./report.tex
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q04$ cat ./report.tex
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名-学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q04$

```

图 12: 新建 report.tex 文件

2.4.2 添加公式、表格以及引用, 完善中文支持

添加带有编号的行间等式使用 `equation`, 在内部使用 `label` 指明标签。添加表格可以先用 `table` 浮动块, 在浮动块内使用 `caption` 制定标题, 其后紧跟 `label` 指明标签, 再使用 `tabular` 插入表格。为了处理模板中原有的中文, 需要使用 `ctex` 宏包。

```

mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git status
位于分支 main
无文件要提交, 干净的工作区
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$ git log --all --graph --decorate --oneline
* 836772a (HEAD -> main) Merge fix: keep mode = safe and add note = reviewed
|
| * 4e249fe (feature-b) Set mode = fast
| * | 3a975bd (feature-a) Set mode = safe
|/
* 81ce968 Initail commit
mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/q03/my-repo$

```

图 13: 完善 report.tex 文件

2.4.3 用 latexmk 编译输出 pdf

由于中文兼容问题, 此处使用 `latexmk -xelatex -halt-on-error ./report.tex` 编译 pdf。latexmk 会自动进行多次编译以保证交叉引用得以正常显示。

Tool Report

王俊杰—25020007122

2026 年 8 月 31 日

1 Result

$$a^2 + b^2 = c^2 \tag{1}$$

勾股定理的数学表示如 (1)。

表 1: 请输入文本	
114514	1919810
8	24

这是一个表格示例 1

图 14: 得到的 report.pdf

3 课后练习

3.1 第 1 题

3.1.1 题目描述

ls 的 -l 选项 (flag) 作用是什么? 运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么?

3.1.2 解题过程

ls 的 -l 选项会输出详细信息, 最前面的 10 个字符中, 第 1 个字符表示文件类型 (- 为普通文件, d 为目录, l 为符号链接, b 为块设备, c 为字符设备, p 为管道, s 为套接字) 之后的 9 个字符每 3 个一组, 分别表示文件所有者, 同用户组, 其他用户的读 (r)、写 (w)、执行 (x) 权限, - 表示无该权限。其中 /tmp 目录具备 t 权限, 表示所有人都能在此处创建文件但只能删除自己创建的文件。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ls -l /
总计 3819628
lrwxrwxrwx    1 root root          7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x    2 root root       4096 Feb 26  2024 bin.usr-is-merged
drwxr-xr-x    3 root root       4096 Aug 22 12:51 boot
dr-xr-xr-x    2 root root       4096 Aug  6  2025 cdrom
-rw-r--r--    1 root root          15 Jul 17 18:59 ctfshow_flag
drwxr-xr-x   19 root root       4400 Aug 30 21:01 dev
drwxr-xr-x  144 root root      12288 Sep  1 17:42 etc
-rw-r--r--    1 root root          25 Mar 10 10:21 flag
drwxr-xr-x    3 root root       4096 Oct 16  2025 home
lrwxrwxrwx    1 root root          7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx    1 root root          9 Nov  2  2025 lib32 -> usr/lib32
lrwxrwxrwx    1 root root          9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x    2 root root       4096 Apr  8  2024 lib.usr-is-merged
lrwxrwxrwx    1 root root         10 Nov  2  2025 libx32 -> usr/libx32
drwx-----   2 root root      16384 Oct 16  2025 lost+found
drwxr-xr-x    3 root root       4096 Oct 16  2025 media
drwxr-xr-x    3 root root       4096 Oct 25  2025 mnt
drwxr-xr-x    3 root root       4096 Aug 12 08:30 opt
dr-xr-xr-x  380 root root          0 Aug 30 21:01 proc
drwx-----  12 root root       4096 Aug 31 02:11 root
drwxr-xr-x   40 root root       1060 Sep  1 17:42 run
lrwxrwxrwx    1 root root          8 Apr 22  2024/sbin -> usr/sbin
drwxr-xr-x    2 root root       4096 Mar 31  2024/sbin.usr-is-merged
drwxr-xr-x   16 root root       4096 Apr 25 10:11 snap
drwxr-xr-x    2 root root       4096 Aug  6  2025 srv
-rw-----    1 root root 3911188480 Oct 16  2025 swap.img
dr-xr-xr-x   13 root root          0 Aug 30 21:01 sys
drwxrwxrwt   24 root root      12288 Sep  1 19:06 tmp
drwxr-xr-x   14 root root       4096 Nov  2  2025 usr
drwxr-xr-x   14 root root       4096 Nov  2  2025 var
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$

```

图 15: ls -l / 输出结果

3.2 第 2 题

3.2.1 题目描述

‘单引号’、”双引号”和 \$‘ANSI 引号’有什么区别？写一条命令，输出一个同时包含字面量 \$、! 和换行符的字符串。

3.2.2 解题过程

单引号保留字符串内除去其自身的字符的字面量；双引号字符串会自动替换 \$var 为其对应的值，进行命令替换和算术替换（替换 \$((expression)) 为 expression 计算所得的值）；ANSI 字符串只会对 \开头的转义序列进行替换。可以执行 echo \$'\$\n!' 利用 ANSI 字符串完成任务。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ echo $'$\n!'  
$  
!  
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$
```

图 16: 命令输出结果

3.3 第 3 题

3.3.1 题目描述

Shell 有三条标准流：stdin (0)、stdout (1)、stderr (2)。运行 ls /nonexistent /tmp，把 stdout 和 stderr 分别重定向到两个文件。你将如何把两者都重定向到同一个文件？

3.3.2 解题过程

> 默认对输出流重定向到下一个参数对应的文件，x>&y 表示将 x 指向的文件重定向到 y 指向的文件，则可以通过 ls /nonexistent /tmp > stdout.log 2> stderr.log 完成第一个任务，通过 ls /nonexistent /tmp > all.log 2>&1 完成第二个任务（先重定向 stdout 再重定向 stderr 到 stdout 指向的文件）。

```
(for_dbg) mrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ls /nonexistent /tmp > stdout.log 2> stderr.log
(for_dbg) mrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat stdout.log
/tmp:
dviptdmx.Kdlu1F
dviptdmx.nLaSxP
dviptdmx.TMygHL
dviptdmx.VQaqmd
dviptdmx.xM4FZx
dviptdmx.z3r11I
dviptdmx.zqh5qa
evince-6433
evince-6490
evince-6621
snap-private-tnp
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-color.service-06JRUY
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-fwupd.service-CHiSOG
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-ModemManager.service-MrbUvN
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-polkit.service-hb0E81
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-power-profiles-daemon.service-71mDtV
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-switcheroo-control.service-DTN11Q
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-logind.service-B1ZGsY
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-oomd.service-KGZ2XS
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-resolved.service-JeHqq1
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-timesyncd.service-SbDwLD
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-upower.service-al0DeN
VMwareDnD
vmware-root
vmware-root_826-2990547547
(for_dbg) mrw@nrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat stderr.log
ls: 无法访问 '/nonexistent': 没有那个文件或目录
```

图 17: 第一条命令的效果

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ls /nonexistent /tmp > all.log 2>&1
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat all.log
ls: 无法访问 '/nonexistent': 没有那个文件或目录
/tmp:
dviPDFmx.Kdlu1F
dviPDFmx.nLa5xP
dviPDFmx.TMygHL
dviPDFmx.VQaqmd
dviPDFmx.xM4FZx
dviPDFmx.z3r11I
dviPDFmx.zqh5qa
evince-6433
evince-6490
evince-6621
snap-private-tmp
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-color.service-06JRUY
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-fwupd.service-CHiSOG
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-ModemManager.service-MrbUvN
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-polkit.service-hb0E81
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-power-profiles-daemon.service-71mDtV
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-switcheroo-control.service-DTN11Q
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-logind.service-B1ZCsY
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-oomd.service-KGZ2XS
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-resolved.service-JeHqQ1
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-systemd-timesyncd.service-SbDwLD
systemd-private-0a8ff351823844d1a8e26a05b9dbc56b-upower.service-a10DeN
VMwareDnD
vmware-root
vmware-root_826-2990547547
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$

```

图 18: 第二条命令的效果

3.4 第 4 题

3.4.1 题目描述

\$? 保存上一条命令的退出状态 (0 表示成功)。&& 仅在前一条成功时执行后一条；|| 仅在前一条失败时执行后一条。写一个一行命令：仅当 /tmp/mydir 不存在时才创建它。

3.4.2 解题过程

可以利用 test 命令，其选项 -d 可以判断下一个参数对应的文件是否为目录，可以使用 [-d /tmp/mydir] || mkdir /tmp/mydir 来实现任务。


```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ls -d /tmp/mydir
ls: 无法访问 '/tmp/mydir': 没有那个文件或目录
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ls -d /tmp/mydir
/tmp/mydir
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ [ -d /tmp/mydir ] || mkdir /tmp/mydir
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ mkdir /tmp/mydir
mkdir: 无法创建目录 "/tmp/mydir": 文件已存在
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$
```

图 19: 命令执行效果

3.5 第 5 题

3.5.1 题目描述

为什么 `cd` 必须是 Shell 内建命令，而不能是独立程序？

3.5.2 解题过程

shell 在执行独立可执行程序时会创建子进程来执行，而子进程仅能修改和父进程共同占有的内核资源（指向同一个文件描述符所对应的偏移，管道等），无法修改父进程的内存空间、环境变量、工作目录、默认权限掩码、进程资源限制。若将 `cd` 设置为独立程序，被执行时只能修改自己子进程的工作目录，而无法修改作为父进程的 shell 的工作目录。所以 `cd` 仅能作为 shell 的内建命令。

3.6 第 6 题

3.6.1 题目描述

在脚本的 `set` 选项（flag）里加入 `-x` 会发生什么？写个简单脚本试试并观察输出。

3.6.2 解题过程

`set -x` 会打开调试模式，执行命令时会先显示提示符 `+`（加号的数量代表是在第几层子 shell 里执行，仅有一个就是当前 shell）和被执行的命令和参数（经过变量替换等）再执行命令。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ vim ./example.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat ./example.sh
set -x
var=114514
echo $var
echo "$var $(ls -l ./example.sh)"
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ chmod +x ./example.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ./example.sh
++ var=114514
++ echo 114514
114514
+++ ls -l ./example.sh
++ echo '114514 -rwxrwxr-x 1 mrw mrw 62 Sep  1 21:02 ./example.sh'
114514 -rwxrwxr-x 1 mrw mrw 62 Sep  1 21:02 ./example.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$

```

图 20: 脚本执行效果

3.7 第 7 题

3.7.1 题目描述

写一条命令，把文件复制为带当天日期的备份文件名（例如 notes.txt → notes_2026-01-12.txt）

3.7.2 解题过程

date 命令可以输出当天日期，则使用命令替换可以实现目的，如 cp notes.txt "notes_\$(date +%Y-%m-%d).txt"。

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ echo 114514 > notes.txt
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat notes.txt
114514
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cp notes.txt "notes_$(date +%Y-%m-%d).txt"
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ find . -name 'notes*'
./notes_2026-09-01.txt
./notes.txt
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat ./notes_2026-09-01.txt
114514
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$

```

图 21: 文件备份效果

3.8 第 8 题

3.8.1 题目描述

使用管道找出你「home 目录」中最常见的 5 种文件扩展名。(提示：组合 find、grep / sed / awk、sort、uniq -c 以及 head)

3.8.2 解题过程

利用 find 命令查找家目录下的含后缀名的文件，利用 sed 和正则表达式匹配并删除最后一个点前的内容，利用 sort 对得到的后缀名排序，使用 uniq -c 统计每行对应的次数，用 sort 命令对次数降序排序，最后用 head 命令输出前 5 行。可以使用 find -type f -name "*.*)" | sed 's/.*\./' | sort | uniq -c | sort -nr | head -5 来完成任务。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ find ~ -type f -name "*.*)" | sed 's/.*\./' | sort | uniq -c | sort -nr | head -n 5
32822 asm
25298 py
21930 h
8907 pyi
7591 pyc
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$
```

图 22: 后缀名统计结果

3.9 第 9 题

3.9.1 题目描述

xargs 会把 stdin 的每一行转换为命令参数。结合 find 和 xargs (不要用 find -exec)，找出目录中所有 .sh 文件，并用 wc -l 统计每个文件行数。加分项：正确处理文件名中的空格。(提示：-print0 和 -0) 参见 man xargs。

3.9.2 解题过程

利用 find 和通配符 *.sh 可以发现后缀为.sh 的文件，但是 find 默认用换行分隔结果而 xargs 默认以空格分隔参数。故 -print0 让 find 以 \0 分隔结果，-0 让 xargs 按 \0 划分参数。可以使用 find . -name '*.sh' -print0 | xargs -0 wc -l 来完成任务。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ echo -e 'echo 1' > test1.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ echo -e 'echo 1\necho 2' > test2.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ find . -name '*.sh' -print0 | xargs -0 wc -l
1 ./test1.sh
2 ./test2.sh
4 ./example.sh
7 总计
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$
```

图 23: .sh 文件统计结果

3.10 第 10 题

3.10.1 题目描述

awk 可以按列值过滤行并改写输出。例如, awk '\$3 /pattern/ {\$4=""}; print}' 会只输出第三列匹配 pattern 的行, 并省略第四列。请写一个 awk 命令: 只输出第二列大于 100 的行, 并交换第一列和第三列。可用这条命令测试: printf 'a 50 x\nb 150 y\nc 200 z\n'

3.10.2 解题过程

awk 默认按空格分隔列, 通过 \$n 表示第 n 行。使用 printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '\$2 > 100 {print \$3, \$2, \$1}' 来完成任务。

```
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 {print $3, $2, $1}'
y 150 b
z 200 c
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$
```

图 24: awk 统计结果

3.11 第 11 题

3.11.1 题目描述

写一个脚本, 接收文件名参数 (\$1), 用 test -f 或 [-f ...] 检查该文件是否存在, 并根据结果输出不同提示。

3.11.2 解题过程

脚本如下所示

Listing 2: IsExistent.sh 的内容

```
1  #!/bin/bash
2  # 判断是否传入1个参数
3  if [ $# -ne 1 ]; then
4      echo "Usage: $0 <csv_file>" >&2
5      exit 1
6  fi
7  # 不存在文件时
8  if [ ! -e $1 ]
9  then
10     echo "$1 doesn't exist" >&2
11     exit 2
12 fi
13 # 存在文件时
14 echo "$1 exists."
```

```

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ vim ./IsExistent.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ chmod +x ./IsExistent.sh
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ cat ./IsExistent.sh
#!/bin/bash
# 判断是否传入1个参数
if [ $# -ne 1 ]; then
    echo "Usage: $0 <csv_file>" >&2
    exit 1
fi
# 不存在文件时
if [ ! -e $1 ]
then
    echo "$1 doesn't exist" >&2
    exit 2
fi
# 存在文件时
echo "$1 exists."

(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ./IsExistent.sh
Usage: ./IsExistent.sh <csv_file>
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ./IsExistent.sh hajimi
hajimi doesn't exist
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$ ./IsExistent.sh example.sh
example.sh exists.
(for_dbg) mrw@mrw-VMware-Virtual-Platform:~/sys-dev-test/week01$

```

图 25: 脚本运行结果

4 commit 记录

Commit 历史截图如图所示，具体 Commit 内容可访问仓库查看。

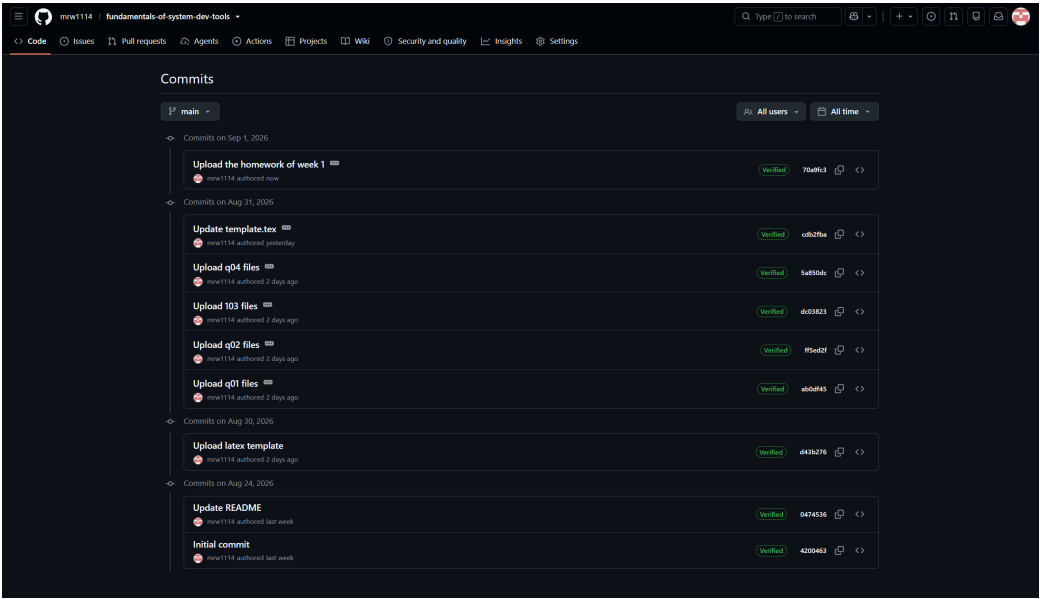


图 26: Commit 历史记录